

CS 35L Software Construction

Lecture 7 – Design & Modeling

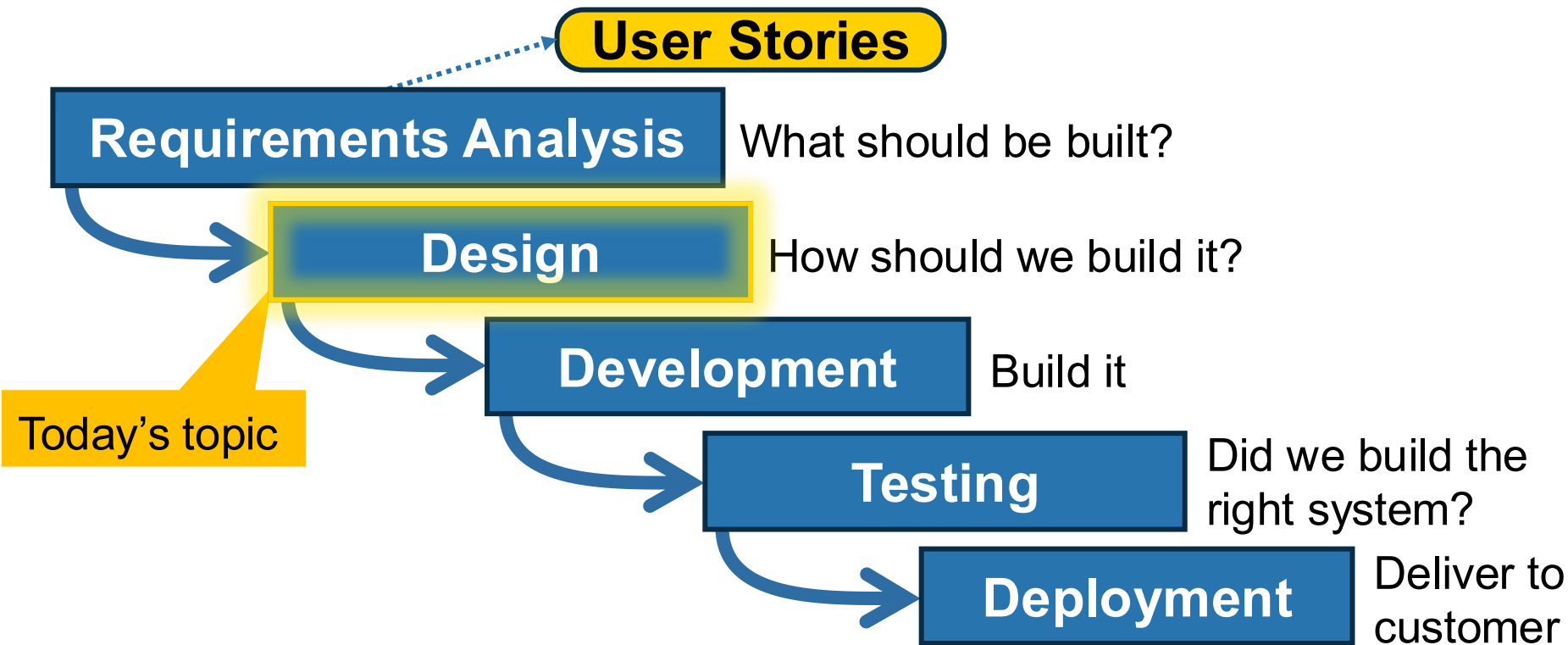
Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



Notes on the Team Project

- You should **start coding now**. We recommend **Pair Programming**
 - two developers use the same computer physically or via Zoom screen sharing to code together.
 - Works especially well for beginners
 - Talking aloud about what you are doing can help you find issues in your design
 - Having someone look over your shoulder means you find bugs more easily
 - Knowledge is exchanged between team members
- Every individual team member will submit a **weekly progress report** on Grade Scope (Fridays starting this week)
 - What have you done this week?
 - How did you apply concepts taught in the lectures to your project?

Recap of Software Engineering Activities



Coding Without Design Considerations Often Turns Very Messy After Some Time

- Early design decisions often become **“load-bearing walls”** of your app (they are hard to change later)
 - E.g., once you start implementing a peer-to-peer architecture, it’s hard to transform into a client-server architecture
 - Spend some time thinking about design before starting to code
 - The more experience you have, the easier it will become to identify which design decisions are “important”



Great Software Engineers Communicate Their Ideas Well



Great Software Engineers Communicate Their Ideas Well



Great Software Engineers Communicate Their Ideas Well

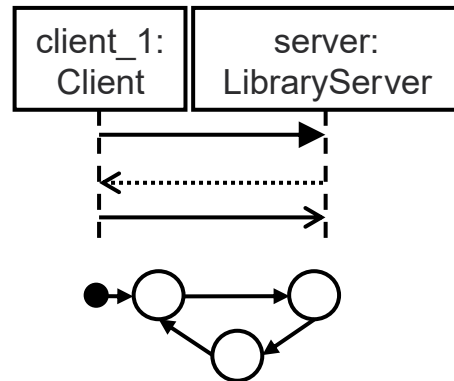
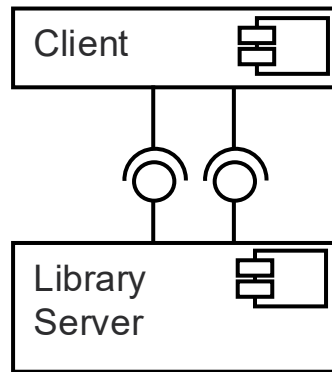
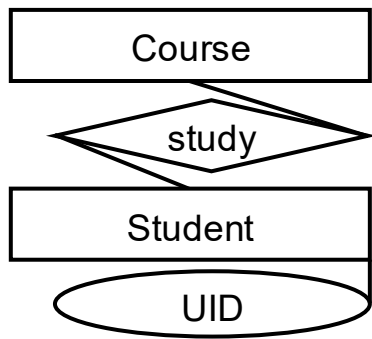
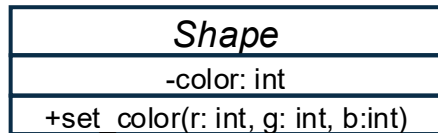


Different Views Visualize Different Aspects of Software

Read more here: [Documenting Software Architectures: Views and Beyond](#) by Clements et al.
(available in UCLA digital library)



Code View / Module View	Data View	Run-Time View / Component- Connector View	Behavioral View
Describes Structure of Code	Describes Structure of Data N	Describes how Run-Time Component are Connected	Describes Interactions

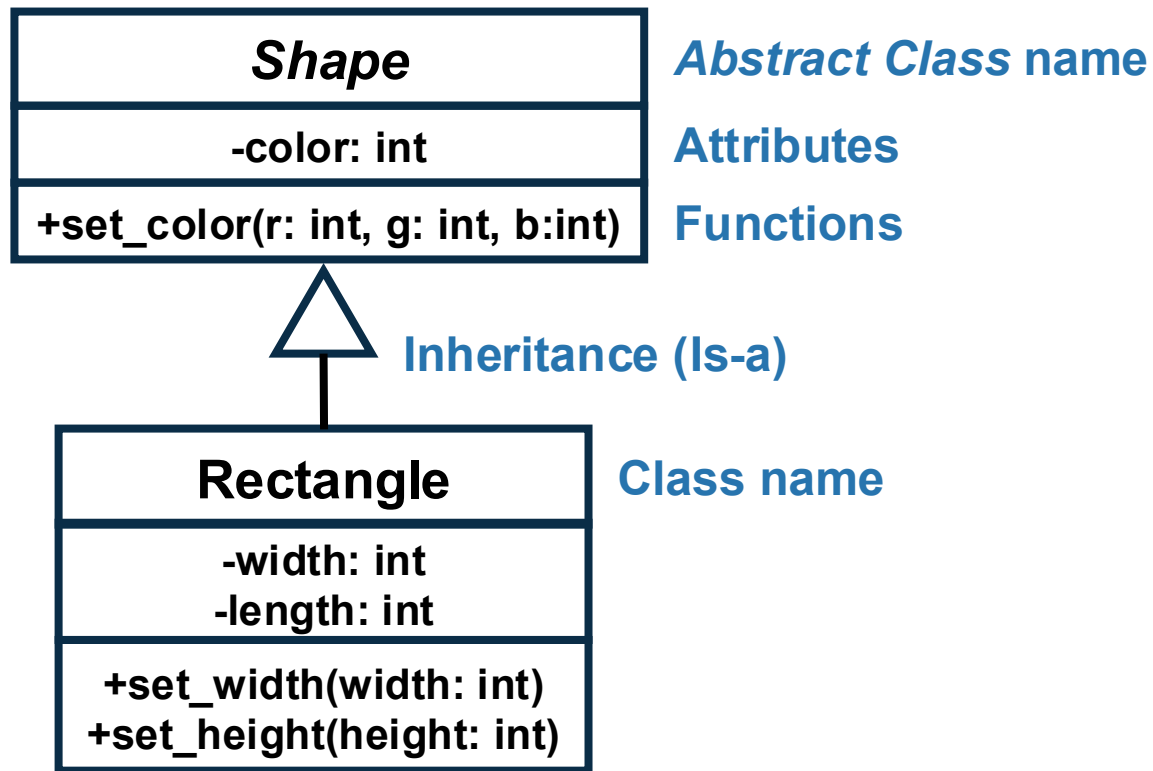


Class Diagrams

Visualize the Code View

Read more here:

https://tobiasduerschmid.github.io/SEBook/uml_class_diagram.html



Visibility:

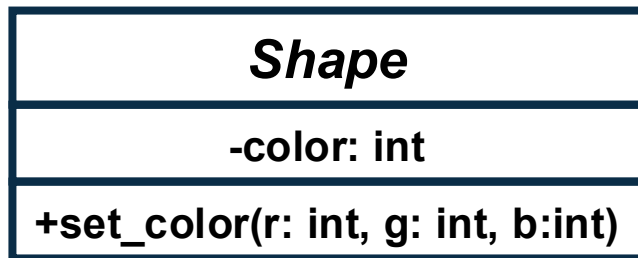
- private
+ public

Class Diagrams

Visualize the Code View

Read more here:

https://tobiasduerschmid.github.io/SEBook/uml_class_diagram.html



```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    def __init__(self):
```

```
        self.__color: int = 0 # private (-color)
```

```
    def set_color(self, r: int, g: int, b: int) -> None:
```

```
        self.__color = (r << 16) | (g << 8) | b
```

```
class Rectangle(Shape):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.__width: int = 0 # private (-width)
```

```
        self.__length: int = 0 # private (-length)
```

```
    def set_width(self, width: int) -> None:
```

```
        self.__width = width
```

```
    def set_height(self, height: int) -> None:
```

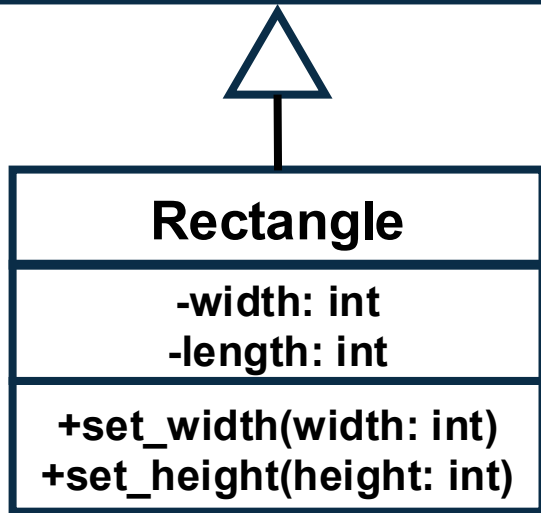
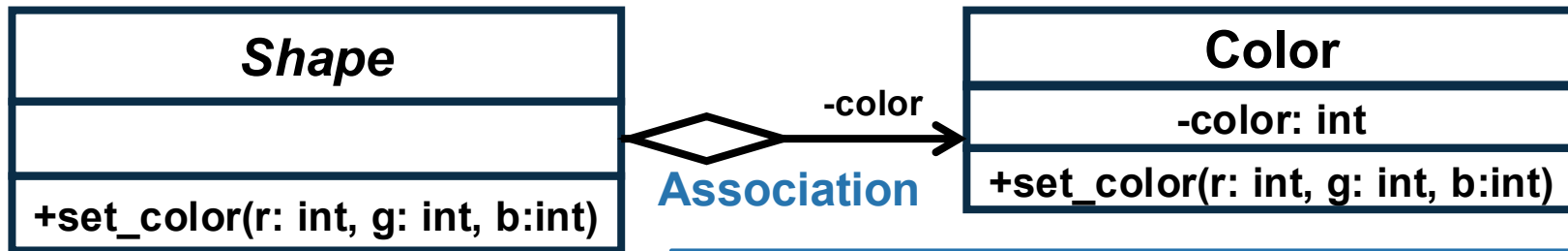
```
        self.__length = height
```

Class Diagrams

Visualize the Code View

Read more here:

https://tobiasduerschmid.github.io/SEBook/uml_class_diagram.html



```
from abc import ABC, abstractmethod

class Color:
    def __init__(self, r: int, g: int, b: int):
        self.set_color()

    def set_color(self, r: int, g: int, b: int) -> None:
        self.r: int = r
        self.g: int = g
        self.b: int = b

class Shape(ABC):
    def __init__(self):
        self.__color: Color = Color(0, 0, 0) # (-color)

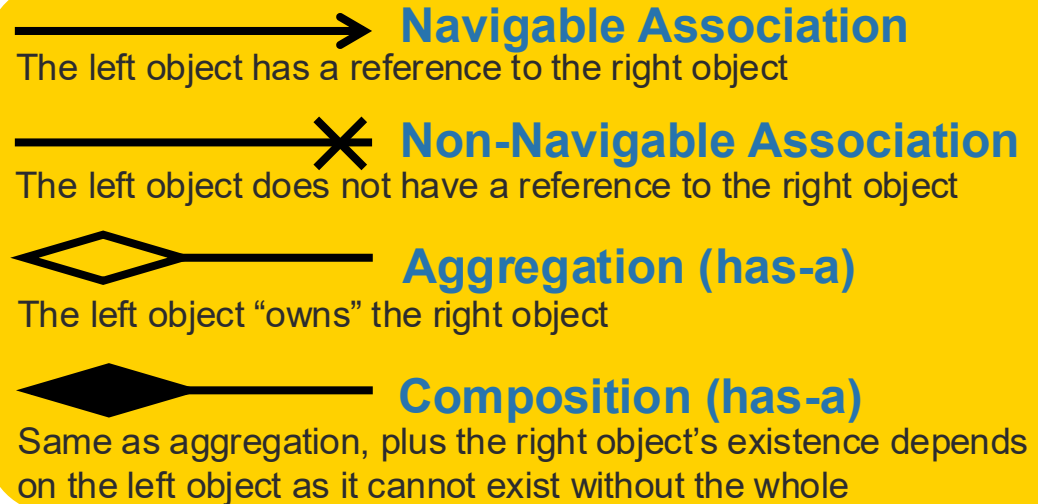
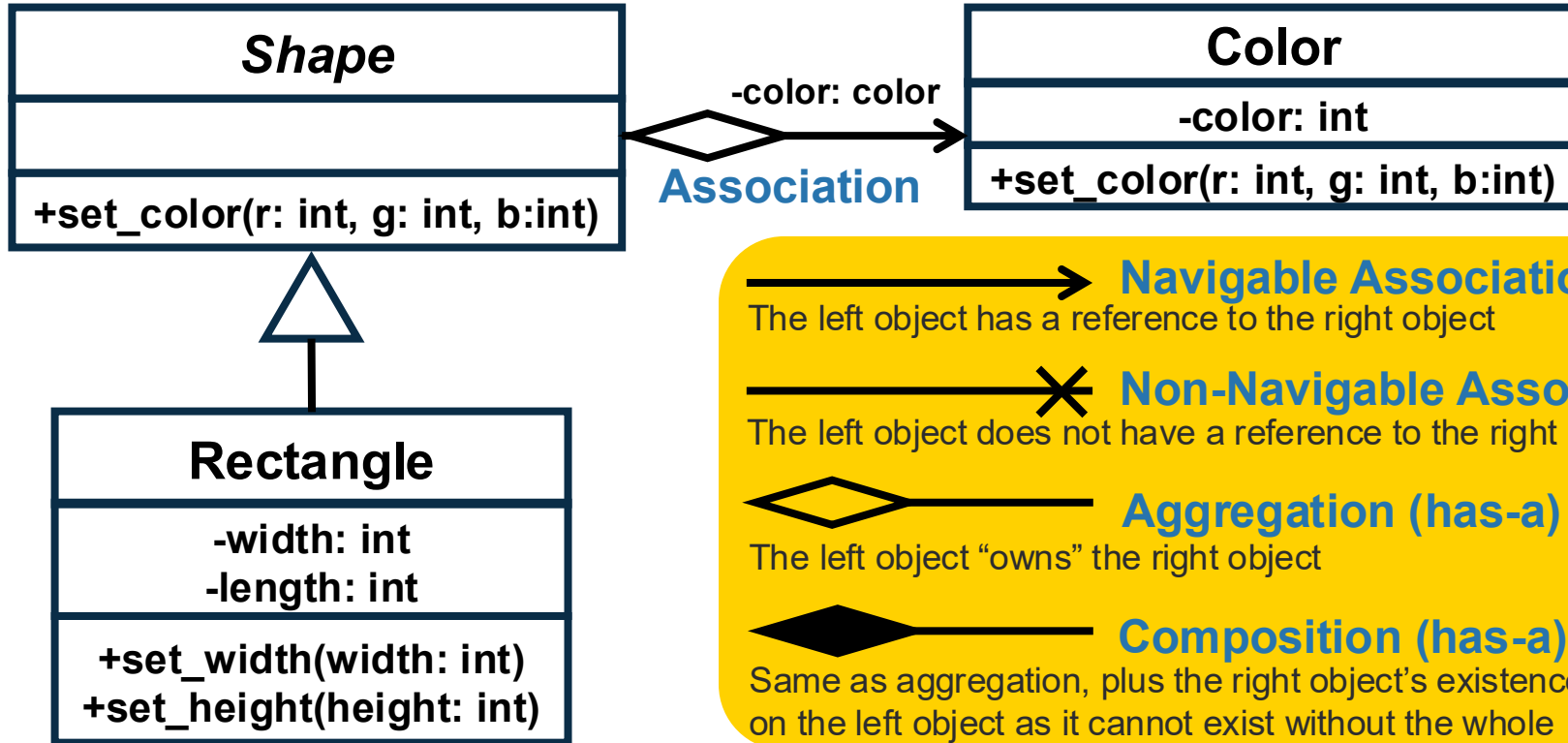
    def set_color(self, color: Color) -> None:
        self.__color = color
```

Class Diagrams

Visualize the Code View

Read more here:

https://tobiasduerschmid.github.io/SEBook/uml_class_diagram.html

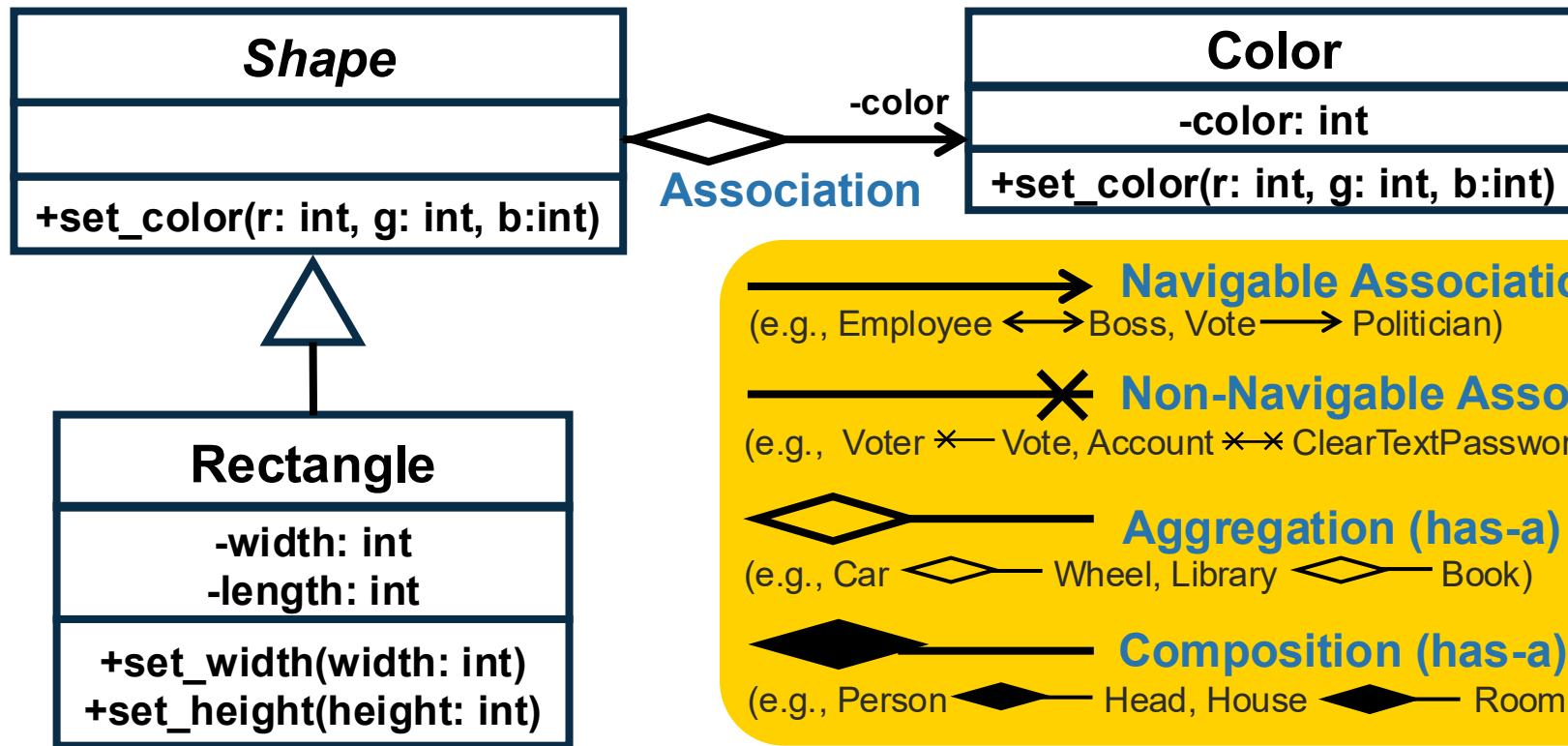


Class Diagrams

Visualize the Code View

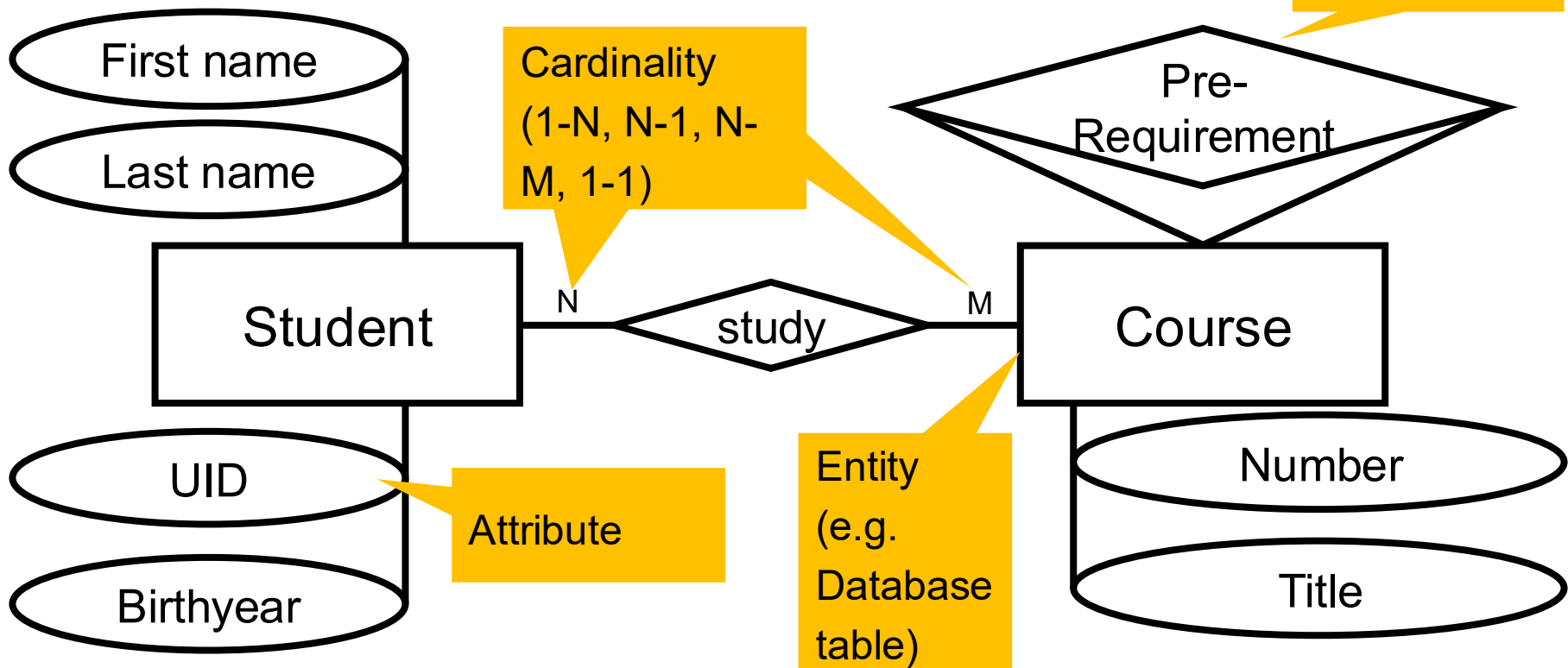
Read more here:

https://tobiasduerschmid.github.io/SEBook/uml_class_diagram.html



Entity-Relationship Diagrams (ERDs)

Visualize the Data View

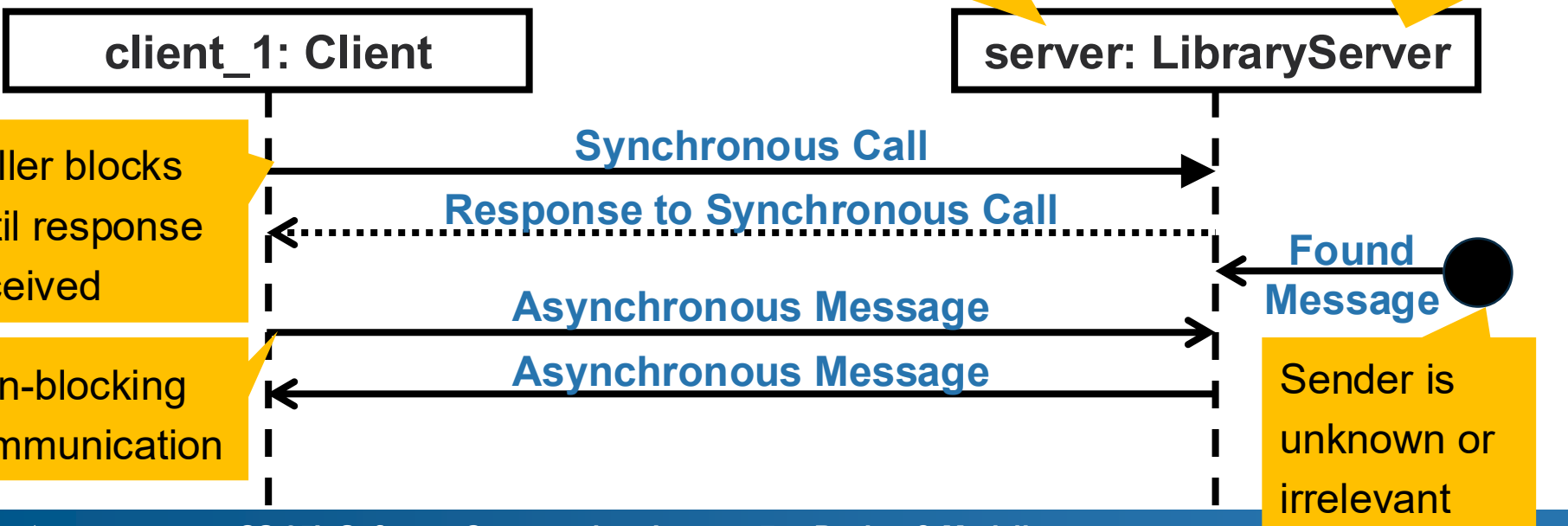


Sequence Diagrams

Visualize the Behavioral View

Read more here
https://tobiasduerschmid.github.io/SEBook/uml_sequence_diagram.html

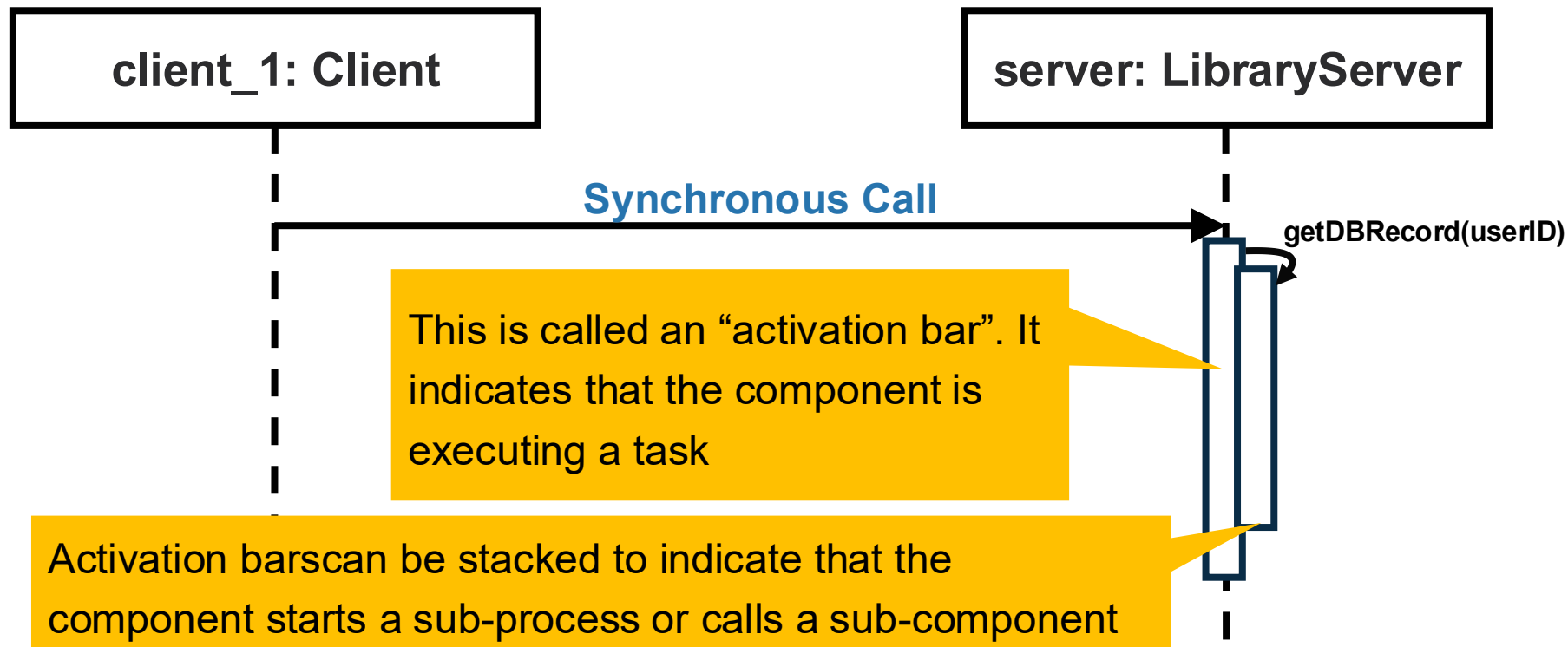
Sequence diagrams show examples of interactions between objects / component instances



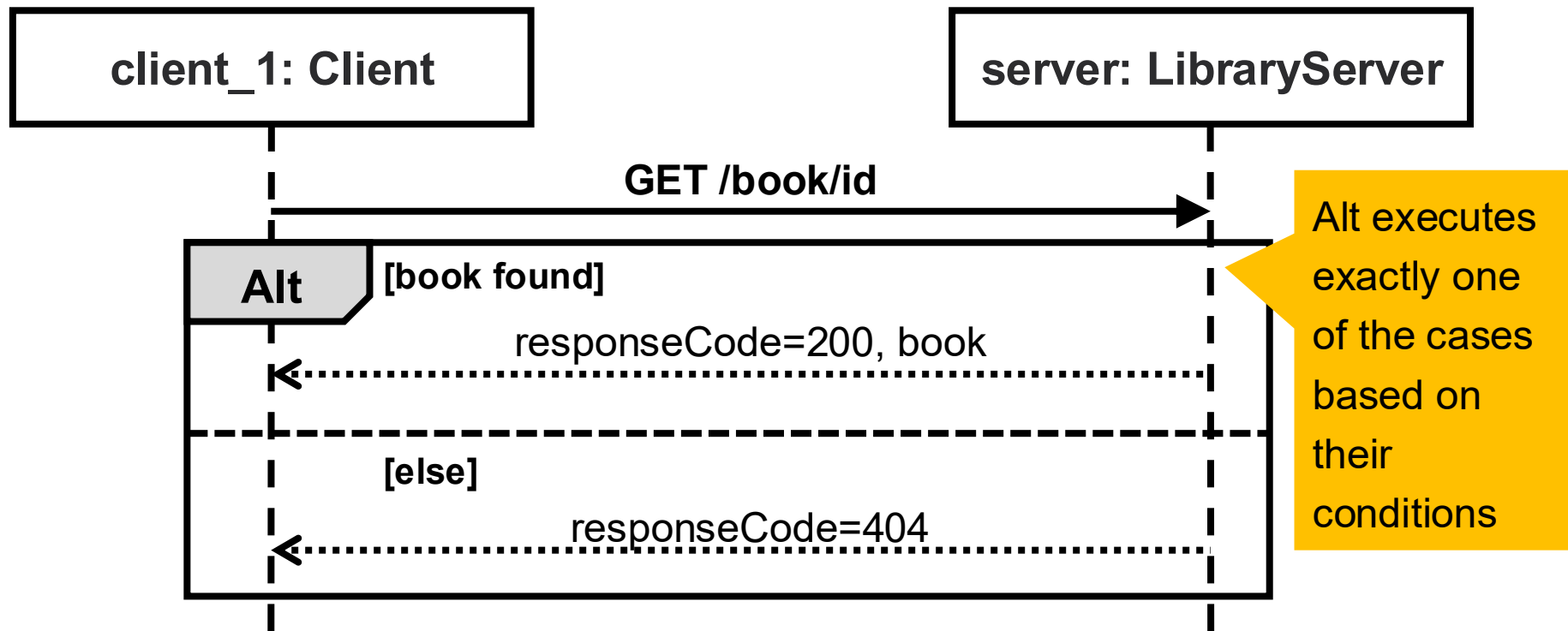
Sequence Diagrams

Visualize the Behavioral View

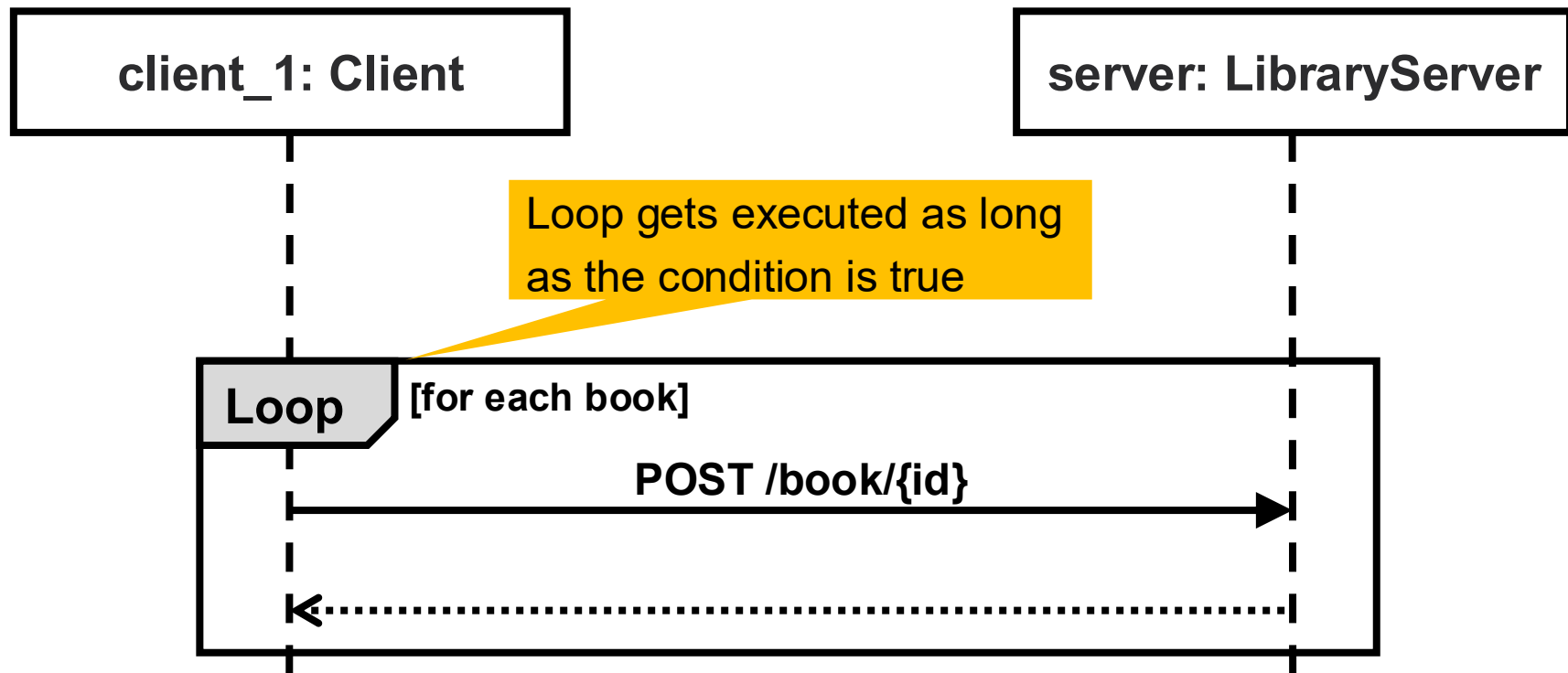
Read more here
https://tobiasduerschmid.github.io/SEBook/uml_sequence_diagram.html



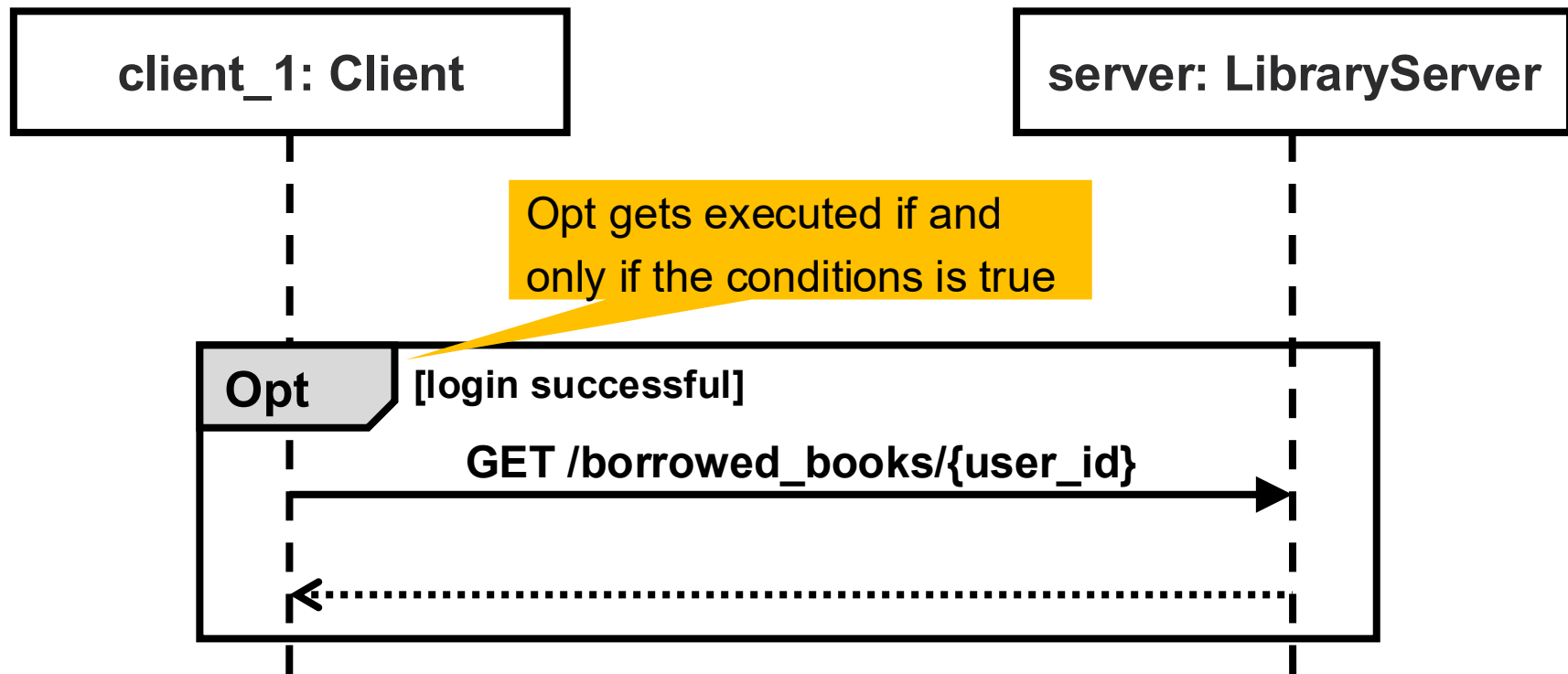
Sequence Diagrams Support Fragments To Show Multiple Execution Paths



Sequence Diagrams Support Fragments To Show Multiple Execution Paths



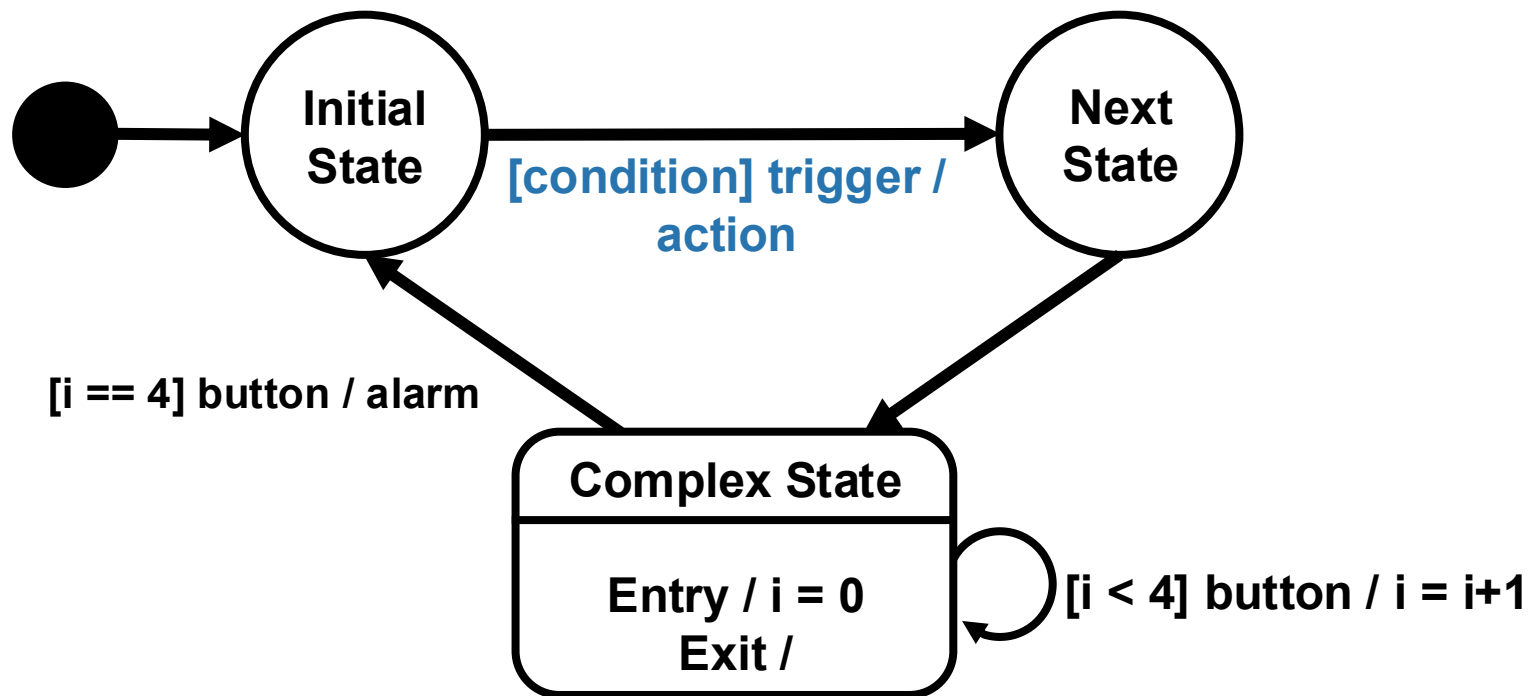
Sequence Diagrams Support Fragments To Show Multiple Execution Paths



State Machine Diagrams

Visualize the Behavioral View

Read more here:
https://tobiasduerschmid.github.io/SEBook/uml_state_diagram.html



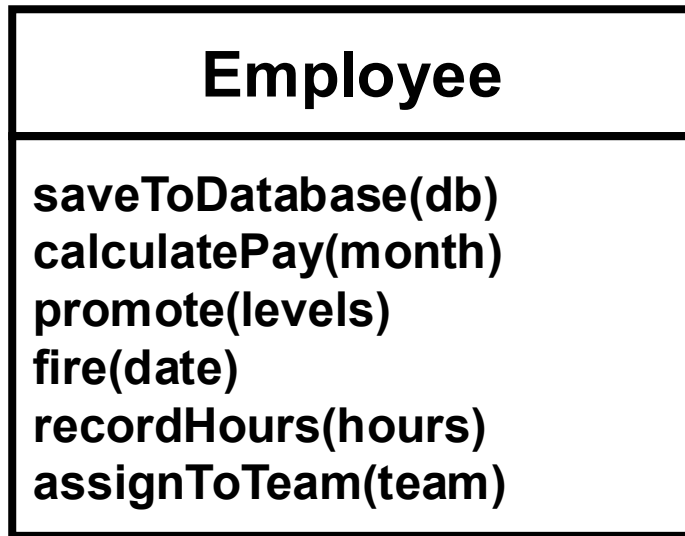
Different Views Visualize Different Aspects of Software

Read more here: [Documenting Software Architectures: Views and Beyond](#) by Clements et al.
(available in UCLA digital library)



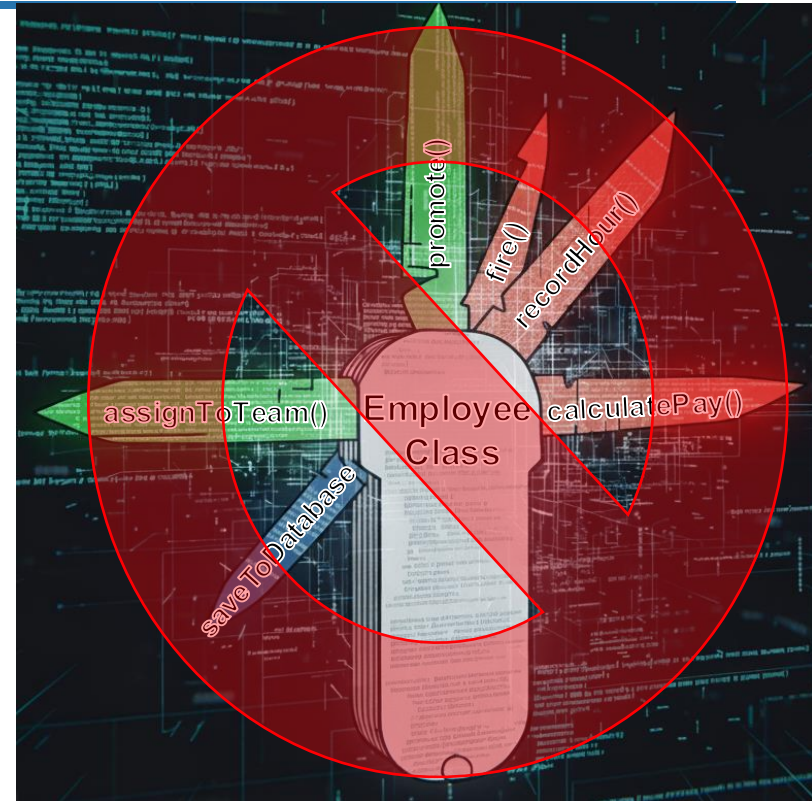
Code View / Module View	Data View	Behavioral View
Describes Structure of Code	Describes Structure of Data	Describes Interactions
Useful for code-level design discussions & documentation (can be inferred automatically from code)	Useful when structuring data base tables	Useful to reason about bottlenecks, messaging protocols, and complex interactions

Is this Class a Good Class? Why or Why Not?

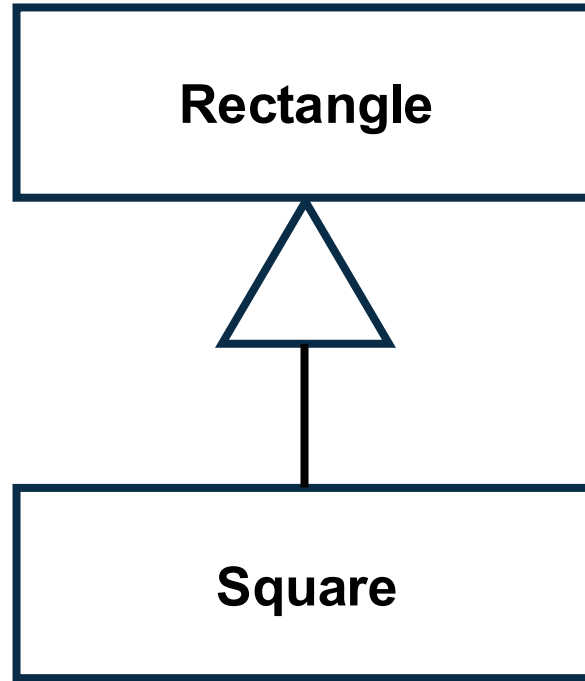


Single Responsibility Principle

- A class should have only one reason to change
- Classes should be designed so that only the needs of one single actor (user from the user story) would require a change in the class

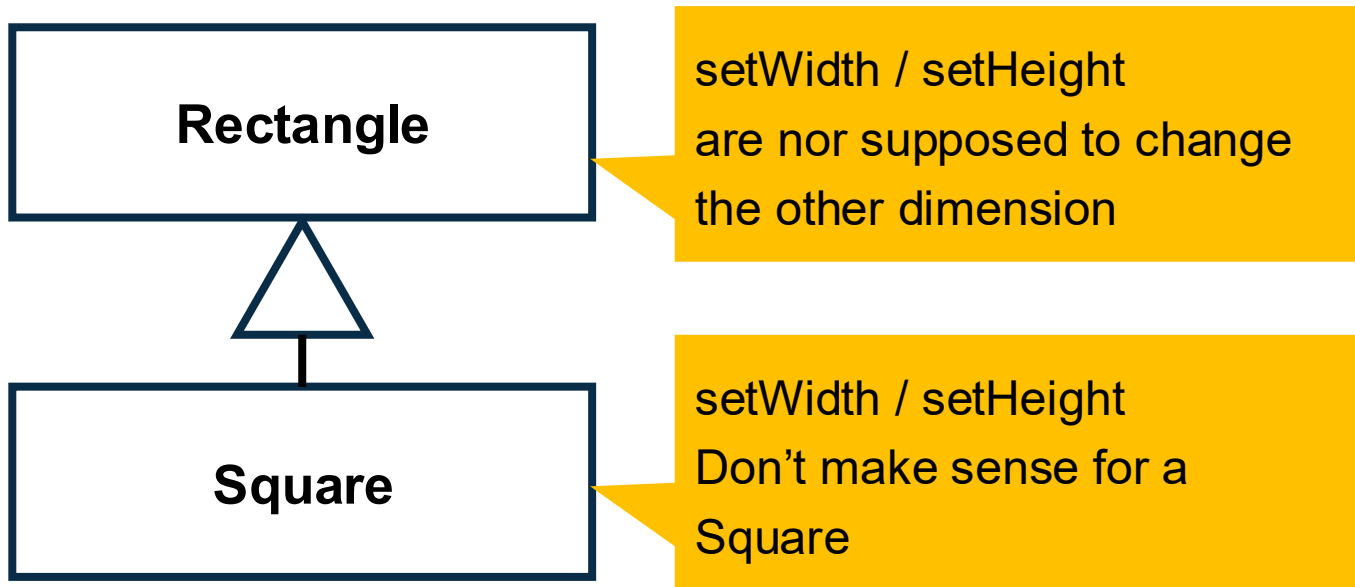


Is this a good class hierarchy? Why or Why Not?



Liskov Substitution Principle

- Subclasses should be substitutable for their super classes without breaking the application's correctness



SOLID Design Principles

- Single Responsibility Principle
- Open-Closed Principle
- Liskov Substitution Principle
- Interface-Segregation Principle
- Dependency Inversion Principle

Open-Closed Principle (OCP)

- A module should be **open for extension** but **closed for modification**
- **Extensions could be:**
 - Subclassing
 - Adding behavior via delegation-based design patterns (e.g., Strategy)
- Modification are:
 - Large changes of the source code

Interface Segregation Principle (ISP)

- **A client module should not be forced to depend on methods it does not use**
- think of this as a Single Responsibility Principle for interfaces
- Split up large interfaces into smaller coherent interfaces

Dependency Inversion Principle (DIP)

- **high-level (abstract) modules should not depend on low-level (concrete) modules.** both should depend on abstractions
- Instead of using information about a concrete class (low-level), you should define an interface (high-level) and only depend on this interface

What is Wrong With This Code?

Talk to your Neighbor(s)

```
function calculateBookOrderTotal(items, discountRate = 0.1, taxRate = 0.05) {
  let subtotal = sumItems(items);
  let discountedTotal = subtotal * (1 - discountRate);
  let finalTotal = discountedTotal * (1 + taxRate);
  console.log(`Book order processed. Subtotal: ${subtotal.toFixed(2)}, Final:
${finalTotal.toFixed(2)}`);
  return finalTotal;
}

// --- Function 2: Calculate Total for Electronics ---
function calculateElectronicsOrderTotal(items, discountRate = 0.05, taxRate = 0.10) {
  let subtotal = sumItems(items);
  let discountedTotal = subtotal * (1 - discountRate);
  let finalTotal = discountedTotal * (1 + taxRate);
  console.log(`Electronics order processed. Subtotal: ${subtotal.toFixed(2)}, Final:
${finalTotal.toFixed(2)}`);
  return finalTotal;
}
```

What is Wrong With This Code?

Too much **Code Duplication**

See

<https://refactoring.guru/smells/duplicate-code>

```
function calculateBookOrderTotal(items, discountRate = 0.1, taxRate = 0.05) {  
  let subtotal = sumItems(items);  
  let discountedTotal = subtotal * (1 - discountRate);  
  let finalTotal = discountedTotal * (1 + taxRate);  
  console.log(`Book order processed. Subtotal: ${subtotal.toFixed(2)}, Final: ${finalTotal.toFixed(2)}`);  
  return finalTotal;  
}  
  
// --- Function 2: Calculate Total for Electronics ---  
function calculateElectronicsOrderTotal(items, discountRate = 0.05, taxRate = 0.10) {  
  let subtotal = sumItems(items);  
  let discountedTotal = subtotal * (1 - discountRate);  
  let finalTotal = discountedTotal * (1 + taxRate);  
  console.log(`Electronics order processed. Subtotal: ${subtotal.toFixed(2)}, Final: ${finalTotal.toFixed(2)}`);  
  return finalTotal;  
}
```

Refactored Code

Avoids Unnecessary Code Duplication

```
function applyDiscountAndTax(items, discountRate, taxRate) {
  let subtotal = sumItems(items);
  let discountedTotal = subtotal * (1 - discountRate);
  let finalTotal = discountedTotal * (1 + taxRate);
  return finalTotal;
}

function calculateBookOrderTotal(items, discountRate = 0.1, taxRate = 0.05) {
  let finalTotal = applyDiscountAndTax(item, discountRate, taxRate);
  console.log(`Book order processed. Subtotal: ${subtotal.toFixed(2)}, Final:
  ${finalTotal.toFixed(2)}`);
  return finalTotal;
}

function calculateElectronicsOrderTotal(items, discountRate = 0.05, taxRate = 0.10) {
  let finalTotal = applyDiscountAndTax(item, discountRate, taxRate);
  console.log(`Electronics order processed. Subtotal: ${subtotal.toFixed(2)}, Final:
  ${finalTotal.toFixed(2)}`);
  return finalTotal;
}
```

What is Wrong With This Code?

Talk to your Neighbor(s)

```
function fetchData(userId) {  
  const url = `${API_BASE_URL}/users/${userId}`;  
  try {  
    const r = await fetch(url);  
    if (!r.ok) {  
      throw new Error(`API request failed with status: ${r.status}`);  
    }  
    const data = await r.json();  
  
    return data;  
  } catch (error) {  
    console.error(`Error fetching user ID ${userId}:`, error.message);  
    throw error;  
  }  
}
```

Naming Smells

See <https://hilton.org.uk/blog/naming-smells>

```
function fetchData(userId) {  
  const url = `${API_BASE_URL}/users/${userId}`;  
  try {  
    What is this URL used for???  
    const r = await fetch(url);  
    if (r) What is r?!!!  
      throw new Error(`API request failed with status: ${r.status}`);  
  }  
  const data = await r.json();  
    What kind of data is this?!!  
  return data;  
} catch (error) {  
  console.error(`Error fetching user ID ${userId}:`, error.message);  
  throw error;  
}
```

Refactored Code better Communicates The Intent & Behavior of the Code

```
function fetchUserData(userId) {  
  const userApiUrl = `${API_BASE_URL}/users/${userId}`;  
  try {  
    const userApiResponse = await fetch(userApiUrl);  
    if (!userApiResponse.ok) {  
      throw new Error(`User API request failed with status:  
                        ${userApiResponse.status}`);  
    }  
    const rawUserData = await userApiResponse.json();  
  
    return rawUserData;  
  } catch (error) {  
    console.error(`Error fetching user ID ${userId}:`, error.message);  
    throw error;  
  }  
}
```

Tutorials Recommended for Content from This Lecture

- [UML Class Diagrams in Python](#)
- [UML Sequence Diagrams in Python](#)
- [SOLID Principles in Python with UML \(Preview Version\)](#)

Please fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

Please summarize three insights you learned about software modelling & design today

Question 2

1 pts

Describe in your own words what the Single Responsibility Principle is and why it is important to create clean, maintainable code.

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slide use images from Flaticon.com (Creators: Freepik)